

Configuración Básica del Contenedor Web Propietario (IIS)

De pantalla de bienvenida a nuestro
propio sitio con TypeScript

Ing. Alberto Manases Turcios Ortiz
Gestión de Servidores Web – Sección A
Miércoles 29 de julio de 2026



Encuadre de la clase

Unidad 1: Creación de contenedores web para la ejecución de servlets

Horario: 1:00 pm – 2:40 pm

Duración: 1 hora 40 minutos

Objetivos de la clase

1. Crear y configurar un sitio web propio en IIS Manager.
2. Configurar bindings (puerto/host) y el documento por defecto.
3. Desplegar una aplicación TypeScript real dentro de IIS usando **iisnode**.

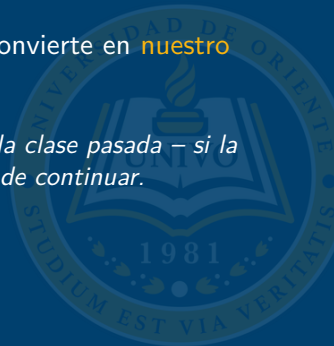


Conexión con la Clase 5

Recordatorio: en la Clase 5 instalamos IIS en nuestra VM con VirtualBox y verificamos que el servicio W3SVC está corriendo.

Hoy: IIS deja de ser una pantalla de bienvenida genérica y se convierte en **nuestro** sitio, corriendo **nuestro** código.

Sigue trabajando en la misma VM de VirtualBox que configuraste en la clase pasada – si la habías apagado o guardado el estado, enciéndela ahora antes de continuar.



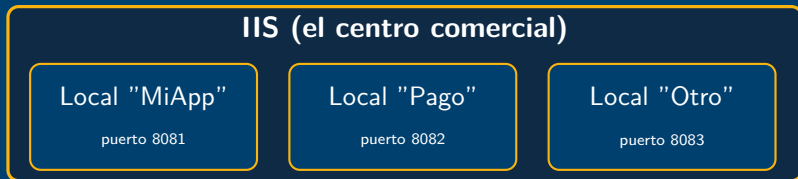


Conceptos clave de IIS Manager

Concepto	Qué es	Analogía
Sitio (Site)	Configuración + carpeta física que IIS expone	Un local comercial dentro de un centro comercial
Application Pool	Proceso aislado donde corre el sitio	Cada local tiene su propia caja registradora – si una se daña, no afecta a las demás
Binding	IP + puerto + host que activa el sitio	El número de local dentro del centro comercial
Documento por defecto	Archivo que se sirve al entrar a la raíz	La vitrina que ven al entrar sin pedir nada específico



El centro comercial, en diagrama



Cada **local** (sitio) tiene su **número** (binding = puerto) y su **propia caja registradora** (Application Pool) – si el local "Pago" se cae, el local "MiApp" sigue atendiendo clientes sin problema.



¿Qué es iisnode?

IIS, por sí mismo, nació pensado para tecnología Microsoft (ASP.NET). **iisnode** es un módulo oficial que le **enseña a IIS a ejecutar aplicaciones Node.js** (y, por lo tanto, nuestro TypeScript compilado) como si fueran nativas.

Esto no es un truco de curso

Empresas reales usan exactamente esta técnica para correr aplicaciones Node dentro de infraestructura Windows ya existente – por ejemplo, un banco que ya tiene cientos de servidores Windows y no quiere migrar todo a Linux solo para usar Node.js.

Hoy vamos a comprobar con las manos que un contenedor "propietario" también puede ejecutar código moderno de "libre distribución" – los dos mundos de la Clase 4 no están tan separados en la práctica.



Paso 0: Instalación de herramientas necesarias

Antes de escribir código, asegurémonos de tener todo instalado en el servidor:

1. Node.js

- Descargar el instalador desde <https://nodejs.org> (versión LTS recomendada).
- Ejecutar el .msi y seguir los pasos por defecto.
- Verificar con:

```
1 node -v    # debe mostrar v18.x o superior
2 npm -v     # debe mostrar 9.x o superior
```

2. TypeScript (global)

```
1 npm install -g typescript
```

Verificar con `tsc -v`.



Paso 1: crear la carpeta del sitio

Todo sitio de IIS necesita una carpeta física en disco donde viven sus archivos:

```
1 New-Item -ItemType Directory -Path "C:\inetpub\miapp"
```

C:\inetpub es la carpeta donde IIS espera, por convención, encontrar los sitios web – es el equivalente de /var/www en un servidor Linux.



Paso 2: nuestra aplicación en TypeScript

Dentro de C:\inetpub\miapp, crear app.ts:

```
1 import * as http from "http";
2
3 const server = http.createServer((req, res) => {
4   res.writeHead(200, { "Content-Type": "text/html; charset=utf-8" });
5   res.end("<h1>Mi primer sitio en IIS (vía iisnode + TypeScript)</h1>");
6 });
7
8 // iisnode inyecta el puerto correcto en esta variable de entorno
9 server.listen(process.env.PORT || 8081);
```

¿Reconocen este patrón? Es el mismo `http.createServer` de la Clase 2 – el contenedor no cambió, solo cambió la casa donde vive.



Paso 2 (cont.): de TypeScript a JavaScript

iisnode ejecuta archivos .js, no .ts – hay que **compilar** primero:

```
1 npm install -g typescript    # si no lo hiciste en el Paso 0
2 tsc app.ts
```

¿Qué significa "compilar", en palabras sencillas?

TypeScript es código pensado para que **nosotros** lo escribamos más seguro (con tipos), pero ningún navegador ni Node.js sabe ejecutarlo directamente. tsc lo **traduce** a JavaScript puro – genera app.js en la misma carpeta. Eso es lo que iisnode realmente corre.



Paso 3: el web.config que conecta IIS con iisnode

web.config es el archivo donde IIS busca **instrucciones especiales** para un sitio – aquí le decimos que use el módulo iisnode para ejecutar app.js:

```
1 <configuration>
2   <system.webServer>
3     <handlers>
4       <add name="iisnode" path="app.js" verb="*" modules="iisnode" />
5     </handlers>
6   </system.webServer>
7 </configuration>
```

*Requiere tener instalado el módulo **iisnode** en el servidor (ya lo hicimos en el Paso 0). Si no se instaló, el sitio no funcionará.*



Paso 4: crear el sitio en IIS

Con PowerShell y el módulo WebAdministration, sin abrir IIS Manager:

```
1 Import-Module WebAdministration
2 New-Website -Name "MiApp" -PhysicalPath "C:\inetpub\miapp" -Port
  8081
```

-Name es el nombre del local en el centro comercial; -PhysicalPath es la carpeta del Paso 1; -Port es el número del local (el binding) que vimos en la tabla de conceptos.



Paso 5: Application Pool dedicado

Buena práctica: nunca compartir el pool por defecto entre sitios distintos.

```
1 New-WebAppPool -Name "MiAppPool"  
2 Set-ItemProperty "IIS:\Sites\MiApp" -Name applicationPool -Value "  
   MiAppPool"
```

Recordando la analogía: si "MiApp" y otro sitio compartieran la misma caja registradora (pool) y "MiApp" se cae por un error, el otro sitio se cae con él. Con pools separados, cada local se sostiene solo.



Paso 6: probar en el navegador

`http://localhost:8081/app.js`

Deben ver: Mi primer sitio en IIS (vía iisnode + TypeScript)

Si no funciona

Revisar en orden: ¿app.js existe (se generó al compilar)? ¿el módulo iisnode está instalado? ¿el sitio quedó con el puerto correcto? ¿el Application Pool está Started?

Si el error persiste, consultar la diapositiva siguiente con los errores más comunes y sus soluciones.



Errores comunes en iisnode

500.19 – Config. bloqueada

HTTP 500.19 - section locked

Causa: IIS bloquea handlers.

Sol: (Admin):

```
1 C:\Windows\system32\inetsrv\appcmd unlock config -section:system.webServer/handlers
```

Para modules:

```
1 C:\Windows\system32\inetsrv\appcmd unlock config -section:system.webServer/modules
```

Luego iisreset.

500.1002 – Permisos/logging

HRESULT: 0x2 - no log file

Causa: AppPool sin escritura o logging activo.

Sol1:

```
1 icacls "C:\inetpub\wwwroot" /grant "IIS_IUSRS:(OI)(CI)F" /T
```

Sol2:

```
1 <iisnode loggingEnabled="false" />
```



El camino completo, de un vistazo

0. **Instalar herramientas:** Node.js, TypeScript, iisnode



1. Crear carpeta C:\inetpub\miapp



2. Escribir app.ts (nuestro contenedor en TypeScript)



3. Compilar: `tsc app.ts` → genera app.js



4. Crear web.config apuntando a iisnode



5. New-Website – registrar el sitio en IIS



6. New-WebAppPool – caja registradora propia



7. Probar en el navegador → "Mi primer sitio..."





Ejercicio individual

1. Crear un sitio con su propio nombre de usuario, en el puerto **8082**, sirviendo una versión modificada de `app.ts` que muestre su nombre completo y la fecha de hoy (usar `new Date()` en TypeScript).
2. Asignarle un Application Pool propio.
3. Capturar pantalla del sitio funcionando y del resultado de:

```
1 Get-WebBinding -Name "<su-sitio>"
```



Pista para el ejercicio

Pista

Sigan **exactamente los 7 pasos** del diagrama anterior, cambiando solo tres cosas: el nombre de la carpeta/sitio, el puerto (**8082** en vez de 8081), y el contenido de `app.ts` (agregar `new Date()` al texto que devuelve).

`new Date()` en TypeScript se comporta igual que en el navegador: crea un objeto con la fecha y hora exactas del momento en que se ejecuta el código – igual que vimos con `getHours()` en la Clase 3.



Verificación de aprendizaje

1. ¿Qué es un binding en IIS?





Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.





Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?





Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?

Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.





Verificación de aprendizaje

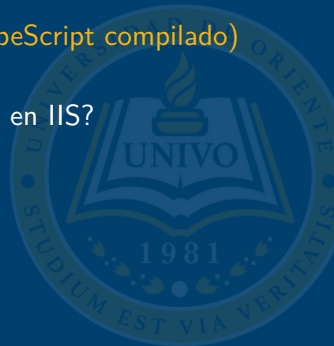
1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?

Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.

3. ¿Por qué hay que compilar el .ts a .js antes de desplegarlo en IIS?





Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

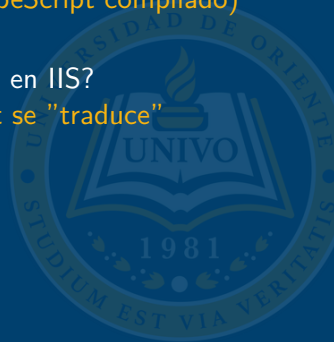
La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?

Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.

3. ¿Por qué hay que compilar el .ts a .js antes de desplegarlo en IIS?

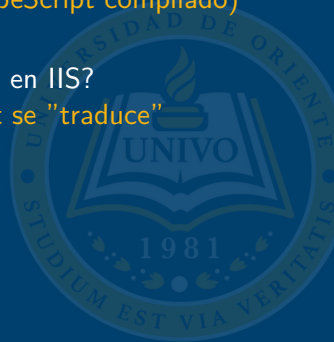
iisnode (y Node.js en general) ejecuta JavaScript; TypeScript se "traduce" (compila) a JavaScript antes de correr.





Verificación de aprendizaje

1. ¿Qué es un binding en IIS?
La combinación IP + puerto + host que activa un sitio.
2. ¿Para qué sirve iisnode?
Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.
3. ¿Por qué hay que compilar el .ts a .js antes de desplegarlo en IIS?
iisnode (y Node.js en general) ejecuta JavaScript; TypeScript se "traduce" (compila) a JavaScript antes de correr.
4. ¿Qué causa el error 500.19 y cómo se soluciona?





Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?

Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.

3. ¿Por qué hay que compilar el .ts a .js antes de desplegarlo en IIS?

iisnode (y Node.js en general) ejecuta JavaScript; TypeScript se "traduce" (compila) a JavaScript antes de correr.

4. ¿Qué causa el error 500.19 y cómo se soluciona?

El bloqueo de la sección <handlers> en web.config; se desbloquea con `appcmd unlock config -section:system.webServer/handlers` y se reinicia IIS.



Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?

Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.

3. ¿Por qué hay que compilar el .ts a .js antes de desplegarlo en IIS?

iisnode (y Node.js en general) ejecuta JavaScript; TypeScript se "traduce" (compila) a JavaScript antes de correr.

4. ¿Qué causa el error 500.19 y cómo se soluciona?

El bloqueo de la sección <handlers> en web.config; se desbloquea con `appcmd unlock config -section:system.webServer/handlers` y se reinicia IIS.

5. ¿Qué causa el error 500.1002 y cómo se soluciona?



Verificación de aprendizaje

1. ¿Qué es un binding en IIS?

La combinación IP + puerto + host que activa un sitio.

2. ¿Para qué sirve iisnode?

Permite que IIS ejecute aplicaciones Node.js (y por tanto TypeScript compilado) como aplicaciones nativas.

3. ¿Por qué hay que compilar el .ts a .js antes de desplegarlo en IIS?

iisnode (y Node.js en general) ejecuta JavaScript; TypeScript se "traduce" (compila) a JavaScript antes de correr.

4. ¿Qué causa el error 500.19 y cómo se soluciona?

El bloqueo de la sección `<handlers>` en `web.config`; se desbloquea con `appcmd unlock config -section:system.webServer/handlers` y se reinicia IIS.

5. ¿Qué causa el error 500.1002 y cómo se soluciona?

Falta de permisos de escritura de la carpeta o logging activado; se soluciona con `icacls` o desactivando `loggingEnabled`.



Resumen de la clase

Componente	Comando/acción clave
Sitio nuevo	<code>New-Website -Name "MiApp"</code> <code>-PhysicalPath "C:\inetpub\miapp"</code> <code>-Port 8081</code>
Application Pool	<code>New-WebAppPool -Name "MiAppPool"</code> y <code>Set-ItemProperty ...</code>
Compilar TS → JS	<code>tsc app.ts</code>
Conectar IIS con Node	<code>web.config</code> con <code><handlers></code> y módulo <code>iisnode</code>
Solución error 500.19	<code>appcmd unlock config</code> <code>-section:system.webServer/handlers</code>
Solución error 500.1002	Permisos (<code>icacls</code>) o <code>loggingEnabled="false"</code>



Tarea para casa

1. Documentar (capturas + breve explicación) la creación completa de su sitio: carpeta, código TypeScript, compilación, `web.config`, sitio, pool, binding, prueba en navegador.
2. Reflexionar por escrito (3-4 líneas): ¿qué fue más complicado, instalar Node.js/Express en Linux (próxima clase) o conectar TypeScript con IIS vía iisnode? (responder **después** de la Clase 7).

Para profundizar (documentación oficial)

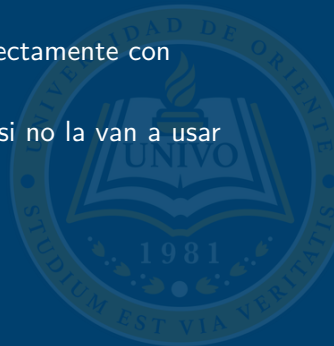
- github.com/Azure/iisnode – repositorio oficial del módulo
- [learn.microsoft.com](https://learn.microsoft.com/en-us/azure/app-service/configure-azure-web-app-azure-functions) – IIS configuration reference



Preparación para la Clase 7

Tema: Instalación de un contenedor web de libre distribución – Node.js + Express (TypeScript) sobre Linux, macOS o Windows.

- Dejar su sitio de IIS funcionando – lo vamos a contrastar directamente con Node.js/Express en la próxima clase.
- Recordar apagar la VM en VirtualBox (o guardar su estado) si no la van a usar antes de la Clase 7, para liberar recursos de su computadora.





Comisión de Acreditación
de la Calidad Académica
UNIVERSIDAD DE ORIENTE - UNIVO
ACREDITADA
2022 - 2027

MUCHAS GRACIAS

Te espero la próxima clase

